

Di01-PW8

November 25, 2025

Grundlagen der Wechselstromlehre

1 Wechselspannung und Wechselstrom

1.1 Periodische Wechselspannung

Eine Wechselspannung ist eine zeitabhängige Spannung

$$U(t)$$

die sich periodisch wiederholt:

$$U(t) = U(t + T)$$

Die Frequenz ist

$$\nu = \frac{1}{T}$$

Zusammenhang zwischen Kreisfrequenz und Frequenz:

$$\omega = 2\pi\nu$$

1.2 Sinusförmige Spannung und Strom

Im Praktikum werden sinusförmige Größen verwendet:

$$U(t) = U_0 \sin(\omega t + \varphi_U)$$

$$I(t) = I_0 \sin(\omega t + \varphi_I)$$

Phasenverschiebung:

$$\Delta\varphi = \varphi_U - \varphi_I$$

1.3 Effektivwerte

Multimeter messen Effektivwerte:

$$U_{\text{eff}} = \frac{U_0}{\sqrt{2}}, \quad I_{\text{eff}} = \frac{I_0}{\sqrt{2}}$$

1.4 Leistung im Wechselstromkreis

Momentanleistung:

$$P(t) = U(t) I(t)$$

Zeitlicher Mittelwert:

$$\langle P \rangle = U_{\text{eff}} I_{\text{eff}} \cos(\Delta\varphi)$$

2 Wechselstromwiderstände (Impedanzen)

2.1 Ohm'scher Widerstand

$$Z_R = R$$

Keine Phasenverschiebung.

2.2 Kondensator

Reeller Wechselstromwiderstand:

$$X_C = \frac{1}{\omega C}$$

Komplexe Impedanz:

$$Z_C = -iX_C = -\frac{i}{\omega C}$$

Der Strom eilt der Spannung um 90° voraus.

2.3 Spule

Reeller Wechselstromwiderstand:

$$X_L = \omega L$$

Komplexe Impedanz:

$$Z_L = iX_L = i\omega L$$

Der Strom hinkt um 90° hinterher.

3 Komplexe Schreibweise

Spannung:

$$\hat{U} = U_0 e^{i(\omega t + \varphi_U)}$$

Strom:

$$\hat{I} = I_0 e^{i(\omega t + \varphi_I)}$$

Komplexe Impedanz:

$$Z = \frac{\hat{U}}{\hat{I}} = \frac{U_0}{I_0} e^{i\Delta\varphi}$$

Vorteil:

Das Ohm'sche Gesetz gilt in komplexer Form:

$$\hat{U} = Z \hat{I}$$

4 Serienschaltung von Impedanzen

Beispiel: Serienschaltung von R und X_C :

$$Z_{\text{ges}} = R - iX_C$$

Betrag:

$$|Z_{\text{ges}}| = \sqrt{R^2 + X_C^2}$$

Spannungen:

$$U_R = IR, \quad U_C = IX_C$$

Gesamtspannung:

$$U_{\text{ges}} = \sqrt{U_R^2 + U_C^2}$$

Phasenverschiebung:

$$\tan \Delta\varphi = \frac{X}{R}$$

5 Bestimmung einer unbekannten Impedanz

In einer Serienschaltung:

$$I = \frac{U_R}{R} = \frac{U_X}{|X|}$$

Daraus:

$$|X| = R \frac{U_X}{U_R}$$

6 Serienschwingkreis (RLC-Kreis)

Gesamtimpedanz:

$$Z = R + i \left(\omega L - \frac{1}{\omega C} \right)$$

Betrag:

$$|Z| = \sqrt{R^2 + \left(\omega L - \frac{1}{\omega C} \right)^2}$$

6.1 Resonanzfrequenz

Resonanzbedingung:

$$\omega L = \frac{1}{\omega C}$$

Resonanzkreisfrequenz:

$$\omega_0 = \frac{1}{\sqrt{LC}}$$

Bei Resonanz:

$$|Z(\omega_0)| = R$$

6.2 Resonanzkurve der Kondensatorspannung

$$U_C(\omega) = U_0 \frac{\omega_0^2}{\sqrt{(\omega_0^2 - \omega^2)^2 + 4\delta^2\omega^2}}$$

Dämpfung:

$$\delta = \frac{R}{2L}$$

Maximum:

$$\omega_{\max} = \sqrt{\omega_0^2 - 2\delta^2}$$

```
[16]: import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

#####
# Messdaten
#####

# Ohmscher Widerstand und seine Unsicherheit (UT61B, ~100 Ohm im
↪ 400-Ohm-Bereich)
R_OHM = 100.0      # gemessener Widerstand [Ohm]
sigma_R = 1.0      # Unsicherheit von R [Ohm] (aus Datenblatt abgeschätzt)

# RC-Messungen
nu_rc = np.array([500, 700, 900, 1100, 1300, 1500, 1700, 1900, 2100, 2300])
UR_rc = np.array([0.45, 0.47, 0.49, 0.51, 0.53, 0.56, 0.60, 0.63, 0.67, 0.70])
UC_rc = np.array([0.80, 0.72, 0.65, 0.59, 0.54, 0.50, 0.46, 0.43, 0.40, 0.37])

# RL-Messungen
nu_rl = np.array([500, 700, 900, 1100, 1300, 1500, 1700, 1900, 2100, 2300])
UR_rl = np.array([0.40, 0.38, 0.37, 0.36, 0.35, 0.34, 0.33, 0.32, 0.31, 0.30])
UL_rl = np.array([0.10, 0.15, 0.21, 0.28, 0.35, 0.42, 0.49, 0.56, 0.63, 0.70])

# RLC-Messungen
nu_rlc = np.array([500, 700, 900, 1100, 1300, 1500, 1700, 1900, 2100, 2300])
UO_rlc = np.array([1.0]*10)
UC_rlc = np.array([0.12, 0.20, 0.38, 0.70, 1.10, 1.25, 1.05, 0.75, 0.50, 0.30])

#####
```

```

# Geräte-Unsicherheiten (UT61B)
#####
# AC-Volt:  $\pm(1.2\% \text{ vom Messwert} + 10 \text{ digits})$  im 4-V-Bereich
# -> REL_U = 0.012, ABS_U = 0.010 V

REL_U = 0.012    # 1.2% laut UT61B Datenblatt
ABS_U = 0.010    # 10 digits -> 0.010 V im 4-V-Bereich

def sigma_U(U):
    """Unsicherheit einer Spannungsmessung mit UT61B im AC-Bereich."""
    return np.sqrt((REL_U * U)**2 + ABS_U**2)

#####
# Theorie-Funktionen
#####

def omega(nu):
    return 2 * np.pi * nu

def Xc_theory(w, C):
    return 1.0 / (w * C)

def Xl_theory(w, L):
    return w * L

def resonance_ratio(w, w0, delta, A):
    num = w0**2
    den = np.sqrt((w0**2 - w**2)**2 + 4 * delta**2 * w**2)
    return A * num / den

#####
# Fit-Qualität:  $\chi^2/\text{DoF}$  und  $R^2$ 
#####

def chi2_per_dof(y, yfit, sigma, n_params):
    """Gewichtetes  $\chi^2/\text{DoF}$  mit bekannten  $\sigma$ ."""
    res = (y - yfit) / sigma
    dof = len(y) - n_params
    return np.sum(res**2) / dof

def r_squared(y, yfit):
    """ $R^2$ , auch für nichtlineare Fits gültig definiert."""
    ss_res = np.sum((y - yfit)**2)
    ss_tot = np.sum((y - np.mean(y))**2)
    return 1 - ss_res/ss_tot

def add_chi2_box(ax, chi2_dof, label=None, x=0.97, y=0.97):

```

```

if label is None:
    txt = rf"$\chi^2/\mathrm{{DoF}} = \{chi2_dof:.3e\}$"
else:
    txt = rf"{label}: $\chi^2/\mathrm{{DoF}} = \{chi2_dof:.3e\}$"
ax.text(x, y, txt,
        transform=ax.transAxes,
        ha="right", va="top",
        fontsize=10,
        bbox=dict(boxstyle="round", facecolor="white", alpha=0.85))

def add_r2_box(ax, R2, label=None, x=0.97, y=0.85):
    if label is None:
        txt = rf"$R^2 = \{R2:.5f\}$"
    else:
        txt = rf"{label}: $R^2 = \{R2:.5f\}$"
    ax.text(x, y, txt,
            transform=ax.transAxes,
            ha="right", va="top",
            fontsize=10,
            bbox=dict(boxstyle="round", facecolor="white", alpha=0.85))

#####
# RC & RL - gemeinsamer Plot gegen mit Unsicherheiten, 2/DoF und R2
#####

def evaluate_rc_rl_omega():

    # ----- RC -----
    w_rc = omega(nu_rc)

    sigma_UR_rc = sigma_U(UR_rc)
    sigma_UC_rc = sigma_U(UC_rc)

    Xc_meas = R_OHM * (UC_rc / UR_rc)

    sigma_Xc = Xc_meas * np.sqrt(
        (sigma_R / R_OHM)**2 +
        (sigma_UC_rc / UC_rc)**2 +
        (sigma_UR_rc / UR_rc)**2
    )

    popt_c, pcov_c = curve_fit(
        Xc_theory, w_rc, Xc_meas,
        p0=[1e-6],
        sigma=sigma_Xc,
        absolute_sigma=True
    )

```

```

C_fit = popt_c[0]
C_err = np.sqrt(np.diag(pcov_c))[0]

Xc_fit = Xc_theory(w_rc, C_fit)
chi2_c = chi2_per_dof(Xc_meas, Xc_fit, sigma_Xc, n_params=1)
R2_c = r_squared(Xc_meas, Xc_fit)

print("\n==== RC-Analyse =====")
print(f"C          = {C_fit:.3e} F ± {C_err:.3e} F")
print(f"chi²/DoF    = {chi2_c:.3e}")
print(f"R²          = {R2_c:.5f}")

# ----- RL -----
w_rl = omega(nu_rl)

sigma_UR_rl = sigma_U(UR_rl)
sigma_UL_rl = sigma_U(UL_rl)

Xl_meas = R_OHM * (UL_rl / UR_rl)

sigma_Xl = Xl_meas * np.sqrt(
    (sigma_R / R_OHM)**2 +
    (sigma_UL_rl / UL_rl)**2 +
    (sigma_UR_rl / UR_rl)**2
)

popt_l, pcov_l = curve_fit(
    Xl_theory, w_rl, Xl_meas,
    p0=[1e-3],
    sigma=sigma_Xl,
    absolute_sigma=True
)
L_fit = popt_l[0]
L_err = np.sqrt(np.diag(pcov_l))[0]

Xl_fit = Xl_theory(w_rl, L_fit)
chi2_l = chi2_per_dof(Xl_meas, Xl_fit, sigma_Xl, n_params=1)
R2_l = r_squared(Xl_meas, Xl_fit)

print("\n==== RL-Analyse =====")
print(f"L          = {L_fit:.3e} H ± {L_err:.3e} H")
print(f"chi²/DoF    = {chi2_l:.3e}")
print(f"R²          = {R2_l:.5f}")

# ----- gemeinsamer Plot -----
fig, ax = plt.subplots(figsize=(8, 5))
ax.set_title(r"Vergleich: $X_C(\omega)$ und $X_L(\omega)$")

```



```

# RC Messwerte + Fehler + Fit
ax.errorbar(
    w_rc, Xc_meas, yerr=sigma_Xc,
    fmt="o", color="blue", label="RC Messwerte"
)
w_fit_rc = np.linspace(min(w_rc), max(w_rc), 500)
ax.plot(
    w_fit_rc, Xc_theory(w_fit_rc, C_fit),
    color="blue", linewidth=2,
    label=r"RC Fit (C={C_fit:.2e} F)"
)

# RL Messwerte + Fehler + Fit
ax.errorbar(
    w_rl, Xl_meas, yerr=sigma_Xl,
    fmt="s", color="red", label="RL Messwerte"
)
w_fit_rl = np.linspace(min(w_rl), max(w_rl), 500)
ax.plot(
    w_fit_rl, Xl_theory(w_fit_rl, L_fit),
    color="red", linewidth=2,
    label=r"RL Fit (L={L_fit:.2e} H)"
)

ax.set_xlabel(r"$\omega$ [1/s]")
ax.set_ylabel(r"$X_C, X_L$ [$\Omega$]")
ax.grid()
ax.legend()

# Statistiken für RC (rechts oben)
add_r2_box (ax, R2_c, label="RC", x=0.3, y=0.80)

# Statistiken für RL (links oben)
add_r2_box (ax, R2_l, label="RL", x=0.6, y=0.80)

plt.tight_layout()
plt.show()

#####
# RLC - Plot gegen mit Unsicherheiten, 2/DoF und R2
#####

def evaluate_rlc_omega():
    w = omega(nu_rlc)
    ratio = UC_rlc / U0_rlc

```

```

sigma_UC = sigma_U(UC_rlc)
sigma_U0 = sigma_U(U0_rlc)

sigma_ratio = ratio * np.sqrt(
    (sigma_UC / UC_rlc)**2 +
    (sigma_U0 / U0_rlc)**2
)

idx = np.argmax(ratio)
w0_start = w[idx]
delta_start = w0_start / 10

popt, pcov = curve_fit(
    resonance_ratio, w, ratio,
    p0=[w0_start, delta_start, 1.0],
    sigma=sigma_ratio,
    absolute_sigma=True
)
w0_fit, delta_fit, A_fit = popt
w0_err, delta_err, A_err = np.sqrt(np.diag(pcov))

ratio_fit = resonance_ratio(w, w0_fit, delta_fit, A_fit)
chi2 = chi2_per_dof(ratio, ratio_fit, sigma_ratio, n_params=3)
R2 = r_squared(ratio, ratio_fit)

print("\n==== RLC-Analyse =====")
print(f"omega0    = {w0_fit:.3e} 1/s ± {w0_err:.3e}")
print(f"delta      = {delta_fit:.3e} 1/s ± {delta_err:.3e}")
print(f"A         = {A_fit:.3e} ± {A_err:.3e}")
print(f"chi²/DoF    = {chi2:.3e}")
print(f"R²         = {R2:.5f}")

fig, ax = plt.subplots(figsize=(7, 5))
ax.set_title(r"RLC-Serie: $U_C/U_0(\omega)$")

ax.errorbar(
    w, ratio, yerr=sigma_ratio,
    fmt="o", label="Messwerte"
)
w_fit = np.linspace(min(w), max(w), 1000)
ax.plot(
    w_fit, resonance_ratio(w_fit, w0_fit, delta_fit, A_fit),
    label="Fit", linewidth=2
)

ax.set_xlabel(r"$\omega$ [1/s]")
ax.set_ylabel(r"$U_C/U_0$")

```

```

ax.grid()
ax.legend()

add_r2_box (ax, R2, x=0.97, y=0.80)

plt.tight_layout()
plt.show()

#####
# Main
#####

if __name__ == "__main__":
    evaluate_rc_rl_omega()
    evaluate_rlc_omega()

```

===== RC-Analyse =====

C = 1.357e-06 F ± 1.448e-08 F

χ^2/DoF = 1.775e+01

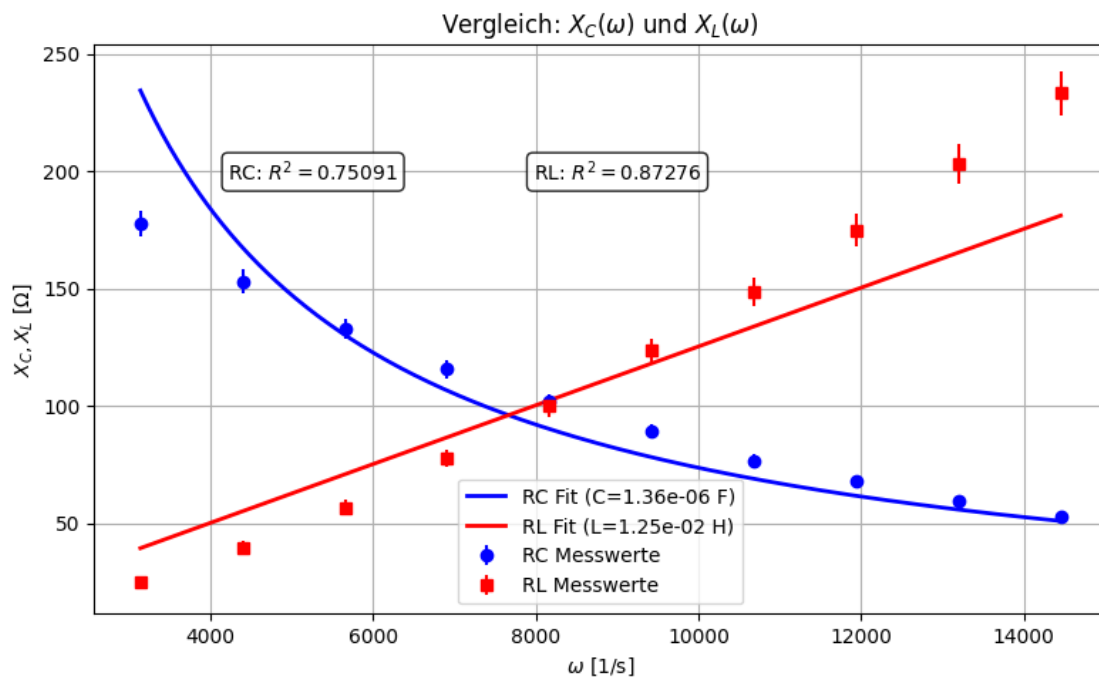
R^2 = 0.75091

===== RL-Analyse =====

L = 1.254e-02 H ± 1.929e-04 H

χ^2/DoF = 1.684e+01

R^2 = 0.87276



===== RLC-Analyse =====

ω_0 = 1.009×10^4 1/s $\pm$ 2.567×10^1

δ = 1.010×10^3 1/s $\pm$ 3.015×10^1

A = 2.843×10^{-1} $\pm$ 3.815×10^{-3}

χ^2/DoF = 1.518×10^2

R^2 = 0.81341

